

LabBench

Study Companion

A hands-on guide to Digital Signal Processing, Control Systems,
Digital Logic, and Communication Systems —
built to be read alongside the LabBench interactive tools.

by **Dhananjay Kumar Seth**
labbench-hub.vercel.app

How to use this guide

Each chapter below pairs a core Electronics & Communication Engineering topic with one of the LabBench tools — free, interactive, browser-based simulators you can open right now. The idea is simple: read a concept, then immediately go turn a knob and watch it happen. Theory sticks far better when you can break it yourself.

Every chapter ends with a short exercise set. Try to answer them by reasoning first, then verify your answer inside the matching LabBench tool.

Contents

- 1. Digital Signal Processing — DSP Signal Lab
- 2. PID Control Systems — PID Control Playground
- 3. Digital Logic & Sequential Circuits — Logic Circuit Simulator + Waveform Viewer
- 4. Communication Systems — Comms Simulator

1. Digital Signal Processing

1.1 — What a signal actually is

A signal is just a value that changes over time — voltage from a microphone, sunlight intensity, your heartbeat. In the analog world it's continuous: at every instant, there's a value. A computer can't store an infinite number of instants, so we **sample** the signal — we measure its value at regular time intervals and keep only those numbers.

1.2 — The sampling theorem (why 44.1 kHz for audio?)

The Nyquist–Shannon sampling theorem says: to perfectly reconstruct a signal, your sample rate must be at least **twice** the highest frequency present in that signal.

$$f_{\text{sample}} \geq 2 \times f_{\text{max}}$$

Human hearing tops out around 20 kHz, so audio is sampled at 44.1 kHz (a bit above the 40 kHz minimum, for margin). Sample below the Nyquist rate and you get **aliasing** — high frequencies fold back and masquerade as lower ones, corrupting the signal irreversibly.

1.3 — The Fourier Transform: from time to frequency

Any signal, no matter how complex, can be described as a sum of simple sine waves at different frequencies and amplitudes. The Fourier Transform is the mathematical tool that takes a signal in the **time domain** (amplitude vs. time) and tells you exactly which frequencies are present and how strong each one is — the **frequency domain**.

The FFT (Fast Fourier Transform) is simply an efficient algorithm for computing this on a computer. A square wave, for instance, isn't a single frequency — it's an infinite sum of odd harmonics (f , $3f$, $5f$, $7f$...) at decreasing amplitude. That's exactly what you'll see spike up in a spectrum analyzer when you feed it a square wave.

*Try it now -> Open **DSP Signal Lab** (dsp-signal-lab.vercel.app) and select the **square** waveform. Watch the spectrum view — you'll see a tall spike at the fundamental frequency, then smaller spikes at 3×, 5×, and 7× that frequency. That's the harmonic series, made visible.*

1.4 — Filters: shaping the frequency content

A filter selectively lets some frequencies through while attenuating others. The four you'll meet constantly:

- **Lowpass (LPF)** — passes frequencies below a cutoff, blocks higher ones. Used to remove hiss/noise or smooth a signal.
- **Highpass (HPF)** — the opposite; passes high frequencies, blocks low ones (e.g. removing DC offset or rumble).
- **Bandpass (BPF)** — passes only a band around a center frequency (e.g. tuning to one radio station).

- **Notch** — the opposite of bandpass; blocks a narrow band (e.g. removing 50/60 Hz mains hum).

A filter's **cutoff frequency** is where it starts attenuating, and its **Q factor** controls how sharp/resonant that cutoff is — a high Q means a narrow, aggressive band; a low Q means a gentler roll-off.

*Try it now -> Open **DSP Signal Lab** (dsp-signal-lab.vercel.app) and add noise to a sine wave, then apply a **lowpass** filter and drag the cutoff down. Watch the high-frequency noise disappear from the spectrum while the fundamental survives.*

Exercises

- 1 A signal contains frequencies up to 8 kHz. What's the minimum sample rate to avoid aliasing?
- 2 You want to isolate a 1 kHz tone buried in broadband noise. Which filter type, and roughly what cutoff(s)?
- 3 Why does a sawtooth wave's spectrum contain *both* odd and even harmonics, unlike a square wave (odd only)? (Hint: symmetry of the waveform about the time axis.)

2. PID Control Systems

2.1 — The problem control theory solves

You have a system with a measurable output (a robot's position, a room's temperature, a drone's altitude) and you want it to track a target — the **setpoint**. The gap between where you are and where you want to be is the **error**: $e(t) = \text{setpoint} - \text{actual}$. A controller's job is to turn that error into a correcting action.

2.2 — The three terms

A PID controller computes its output as a sum of three independent responses to the error signal:

$$u(t) = K_p \cdot e(t) + K_i \cdot (\text{running sum of } e(t)) + K_d \cdot (\text{rate of change of } e(t))$$

- **Proportional (K_p)** — reacts to the *current* error. Bigger error, bigger correction. Alone, it always leaves some steady-state offset and tends to overshoot/oscillate.
- **Integral (K_i)** — reacts to the *accumulated* error over time. This is what eliminates steady-state error — even a tiny persistent offset keeps building the integral until it forces a correction. Too much K_i , though, causes overshoot and slow oscillation ("integral windup").
- **Derivative (K_d)** — reacts to *how fast* the error is changing. It acts as a brake: if the error is closing quickly, D pulls back the output to prevent overshoot. It adds damping and stability, but amplifies noise if set too high.

*Try it now -> Open **PID Control Playground** (pid-control-playground.vercel.app) and load the **P-only** preset and watch the robot oscillate around the line forever. Then load **PD** and watch the derivative term damp that oscillation almost immediately.*

2.3 — Reading a step response

The classic way to evaluate a tuned controller is a step response: suddenly change the setpoint and watch how the output responds. Key things to look for:

- **Rise time** — how fast it first reaches the target.
- **Overshoot** — how far past the target it swings before settling.
- **Settling time** — how long until it stays within a small band of the target.
- **Steady-state error** — the gap that remains forever, if any.

Tuning is fundamentally a trade-off: push K_p up for faster response and you'll usually get more overshoot; add K_d to tame the overshoot but too much makes the system sluggish and noise-sensitive; add K_i to kill steady-state error but too much causes slow, creeping oscillation.

Exercises

- 1 A robot consistently settles 2 cm to the left of the line and stays there. Which term is missing or too weak?

- 2 A drone's altitude controller oscillates wildly and never settles. Which term would you increase first, and why?
- 3 Why does a pure integral controller ($K_p=K_d=0$) respond very slowly at first?

3. Digital Logic & Sequential Circuits

3.1 — Combinational vs. sequential

A **combinational** circuit's output depends only on its *current* inputs — an AND gate, an adder, a multiplexer. Feed it the same inputs twice, you get the same output, no memory involved. A **sequential** circuit's output depends on current inputs *and* its history — it has memory. Flip-flops, counters, and shift registers are all sequential.

*Try it now -> Open **Logic Circuit Simulator** (logic-circuit-sim.vercel.app) and load the **SR Latch** example. Notice the two NOR gates feed into each other — that feedback loop is literally what gives the circuit memory.*

3.2 — The flip-flop: one bit of memory

A D flip-flop is the simplest useful memory element: on every rising edge of a clock signal, it "captures" whatever value is on its D input and holds it on its Q output until the next rising edge — regardless of what D does in between. This is called **edge-triggered** behavior, and it's the foundation of every register, counter, and state machine in digital design.

*Try it now -> Open **Waveform Viewer** (waveform-viewer-theta.vercel.app) and select the **D Flip-Flop** preset and click a cell in the D row to pulse it high for one clock period. Watch Q go high exactly one period later — that delay is edge-triggered capture, made visible.*

3.3 — Building blocks from flip-flops

- **Clock divider (T flip-flop)** — toggles every rising edge, so its output frequency is exactly half the clock's. Chain N of them and you divide by 2^N .
- **Counter** — a chain of flip-flops where each bit increments in binary on every clock edge (0000 -> 0001 -> 0010 -> ...). The classic "ripple" pattern: bit 0 toggles every cycle, bit 1 every 2 cycles, bit 2 every 4, and so on.
- **Shift register** — a chain of flip-flops where each stage passes its value to the next on every clock edge, and a new bit enters at the start. Used for serial-to-parallel conversion.

*Try it now -> Open **Waveform Viewer** (waveform-viewer-theta.vercel.app) and switch to the **4-Bit Synchronous Counter** preset (no input needed) and watch Q0 toggle every cycle, Q1 every 2 cycles, Q2 every 4 — the binary ripple pattern, laid out as a timing diagram.*

3.4 — Truth tables and Boolean minimization

Every combinational circuit has a truth table: every possible input combination, mapped to its output. Two circuits with identical truth tables are functionally interchangeable, even if built from completely different gates — this is what lets you simplify a circuit (fewer gates, less power, less delay) without changing its behavior.

*Try it now -> Open **Logic Circuit Simulator** (logic-circuit-sim.vercel.app) and load the **Half Adder** example and read its truth table. Notice $Sum = A \oplus B$ and $Carry = A \wedge B$ — the entire adder is just those two gates.*

Exercises

- 1 How many flip-flops do you need to build a counter that counts from 0 to 63?
- 2 In a 4-bit shift register, how many clock cycles does it take for a bit entering at SIN to reach Q3?
- 3 Write the truth table for a 2-to-1 multiplexer with inputs A, B and select line S (output = A when S=0, B when S=1).

4. Communication Systems

4.1 — Why modulate at all?

Your voice, as an electrical signal, sits in the 300 Hz – 3 kHz range. You can't broadcast that directly through the air efficiently — antennas need to be a meaningful fraction of the wavelength, and everyone's voice would collide on the same frequencies. **Modulation** shifts your information onto a much higher-frequency carrier wave, which can be efficiently radiated and separated from other signals by frequency.

4.2 — AM and FM

Amplitude Modulation (AM) varies the carrier's amplitude in proportion to the message signal, leaving frequency constant.

$$s(t) = [1 + k \cdot m(t)] \cdot \cos(2 \cdot \pi \cdot f_c \cdot t)$$

k is the modulation index. Push it above 1.0 and you get **over-modulation** — the envelope crosses zero, and a simple envelope detector can no longer recover the original message without distortion.

Frequency Modulation (FM) instead varies the carrier's *frequency* in proportion to the message, keeping amplitude constant — which makes FM far more resistant to amplitude noise (most real-world interference), which is exactly why FM radio sounds cleaner than AM.

*Try it now -> Open **Comms Simulator** (comms-simulator-pi.vercel.app) and on the Analog tab, push the AM modulation index past 1.0 and watch the envelope pinch and cross zero — that's over-modulation, visualized.*

4.3 — Digital modulation: BPSK, QPSK, and QAM

Instead of a continuous message, digital modulation maps bits onto discrete symbols, each a specific amplitude/phase combination of the carrier — visualized as a point on a **constellation diagram** (in-phase I on the x-axis, quadrature Q on the y-axis).

- **BPSK** — 2 points (1 bit/symbol), maximally far apart. Most robust to noise, lowest data rate.
- **QPSK** — 4 points (2 bits/symbol). Doubles the data rate at the same symbol rate, needs somewhat higher SNR.
- **16-QAM** — 16 points in a grid (4 bits/symbol). Much higher data rate, but the points are packed closer together, so it needs significantly higher SNR to avoid errors.

This is the fundamental bit-rate-vs-robustness trade-off in every wireless standard you've ever used: higher-order modulation packs more bits per symbol but demands a cleaner channel.

*Try it now -> Open **Comms Simulator** (comms-simulator-pi.vercel.app) and on the Constellation tab, switch between BPSK and 16-QAM at the same SNR. Watch the 16-QAM points blur together and produce visible symbol errors (red dots) while BPSK stays clean.*

4.4 — Bit Error Rate and the noisy channel

Real channels add noise (commonly modeled as AWGN — Additive White Gaussian Noise). The **Bit Error Rate (BER)** is the fraction of received bits that come out wrong, and it drops sharply as signal-to-noise ratio (SNR, often expressed as E_b/N_0 in dB) increases. This relationship follows a well-known curve derived from the Q-function, and matching a real system's measured BER against that theoretical curve is one of the standard ways engineers verify a communication link is working correctly.

*Try it now -> Open **Comms Simulator** (comms-simulator-pi.vercel.app) and on the BER Curve tab, compare BPSK and 16-QAM — notice BPSK needs roughly 4 dB less SNR than 16-QAM to hit the same bit error rate. That gap is the real-world price of higher data rate.*

Exercises

- 1 Why is FM generally more noise-resistant than AM for the same transmit power?
- 2 A link needs to carry 4 bits per symbol. Which modulation scheme from this chapter fits, and what's the trade-off versus BPSK?
- 3 If you halve the transmit power (roughly -3 dB in SNR), what happens to BER, qualitatively?

Keep building.

That's the core of all four topics — but the real learning happens back inside the tools, breaking things on purpose and watching why they broke. All five LabBench tools are free, forever, no account required.

labbench-hub.vercel.app

Thank you for supporting LabBench.